

Name of the Student	Yoga Madha A
Reg. No	192425058
GitHub Link	https://github.com/YOGAMADHA/MLA0405-DEEP-LEARNING

SIMATS ENGINEERING

CO4 - ASSESSMENT TOOL 1

Design and Develop Model

COURSE NAME: Deep Learning
SLOT : D

COURSE CODE: MLA0405

COURSE OUTCOME COVERED

CO 4: Students will be able to understand and apply probabilistic graphical models including Bayesian Networks, Markov Random Fields, Hidden Markov Models, and Conditional Random Fields. They will learn how to represent complex probabilistic relationships among variables and perform inference and learning in graphical models. Students will be able to use these models for reasoning under uncertainty and solving real-world decision-making problems. BL-6

Course Outcome Weightage: 10%

Duration: 90 minutes

Assessment Tool Weightage: 30%

Mark : 25

Code of Conduct:

I Yoga Madha A-192425058 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.



Signature of the student

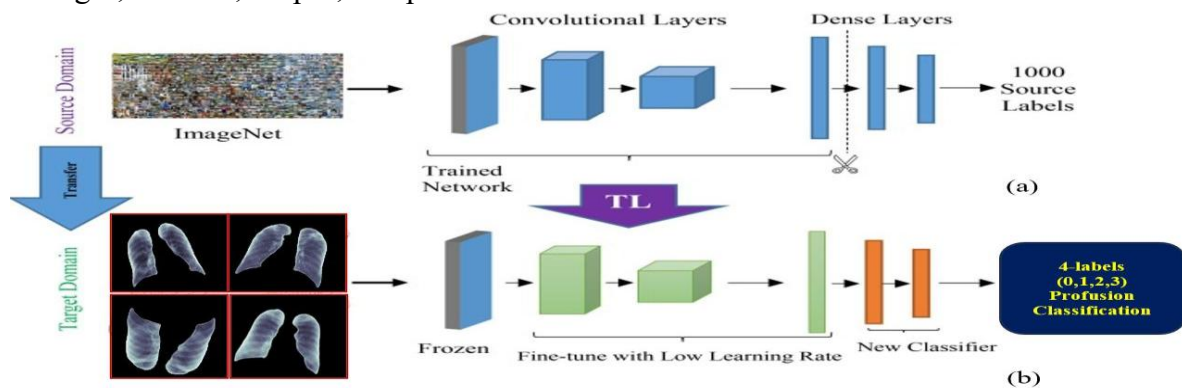
Name of the student: Yoga Madha A

Criteria	Problem Understanding & Requirement Analysis (2)	Model Architecture & Design (2.5)	Application of Deep Learning Techniques (2.5)	Model Development & Implementation Strategy (2)	Performance Analysis & Justification (1)	Total (10)
Weightage	20 %	30%	25%	15 %	10 %	100 %
Q1						
Q2						
Q3						
Q4						
Q5						
Total						

1. Design and develop a model for a hospital has only 2,000 labeled X-ray images for classifying images into normal and abnormal categories. A pre-trained ResNet model is available. Design a CNN-based transfer learning model by identifying which layers should be frozen, which layer should be replaced, and how the model should be trained.

Answer:

A **CNN-based Transfer Learning model using ResNet** is suitable because the hospital has only 2,000 labeled X-ray images. Training a deep CNN from scratch with such a small dataset can lead to overfitting. A pre-trained ResNet can reuse previously learned visual features such as edges, textures, shapes, and patterns.



Proposed Architecture

X-ray Image → Preprocessing → Pre-trained ResNet → Global Average Pooling → New Fully Connected Layer → Sigmoid → Normal/Abnormal

1. Input and Preprocessing

- Resize X-ray images to the input size required by ResNet, commonly 224×224 pixels.
- Normalize pixel values.
- Apply data augmentation such as:
 - Rotation
 - Horizontal flipping where medically appropriate
 - Zooming
 - Small translations
- Divide the dataset into training, validation, and testing sets.

2. Frozen Layers

The initial convolutional layers of ResNet should initially be **frozen**.

These layers have already learned general visual features such as:

- Edges
- Lines
- Textures
- Shapes
- Basic patterns

Freezing them:

- Reduces training time.
- Reduces the number of trainable parameters.
- Helps prevent overfitting.
- Allows effective learning from a small dataset.

3. Replace the Final Layer

The original ResNet classification layer is designed for its original large-scale classification task. Therefore, the final fully connected classification layer should be removed/replaced.

A new classification head can contain:

ResNet Feature Extractor → Global Average Pooling → Dense Layer → Dropout → Sigmoid Output

For binary classification:

- **Sigmoid output = 1:** Abnormal
- **Sigmoid output = 0:** Normal

4. Training Process

Initially:

- Freeze the ResNet base.
- Train only the newly added classification layers.
- Use **binary cross-entropy loss**.
- Use an optimizer such as **Adam**.

After initial training, some of the deeper ResNet layers can be **unfrozen** and fine-tuned using a very small learning rate.

5. Final Model

Component	Action
Input	X-ray image
Early ResNet layers	Frozen
Middle/deep ResNet layers	Initially frozen, optionally fine-tuned
Original classification layer	Removed
New classification head	Added
Output	1 neuron with Sigmoid
Loss	Binary Cross-Entropy
Optimizer	Adam
Prediction	Normal / Abnormal

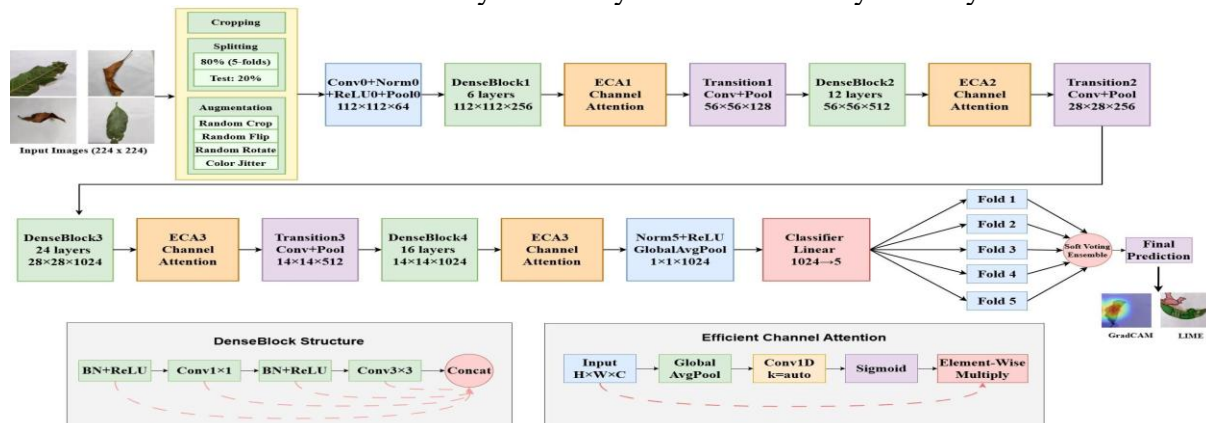
Conclusion

Transfer learning with ResNet is appropriate because the limited 2,000-image dataset can benefit from features learned from a large pre-training dataset. Freezing the early layers and replacing the final classification layer provides an efficient model while reducing overfitting.

2. An agricultural organization wants to identify five types of plant diseases from leaf images. Since the dataset contains limited training samples, the organization wants to reuse features efficiently. Design a DenseNet-based classification model and explain how dense connections can help the model reuse features during classification.

Answer:

A **DenseNet-based image classification model** can be designed to identify five different plant diseases. DenseNet is particularly useful when training data is limited because its dense connections allow features learned by earlier layers to be reused by later layers.



Proposed Architecture

Leaf Image → Preprocessing → DenseNet Feature Extractor → Global Average Pooling → Dropout → Dense Layer → Softmax → 5 Disease Classes

1. Input Processing

Leaf images are:

- Resized to a fixed size.
- Normalized.
- Augmented using rotation, flipping, cropping, and zooming.

This increases the effective diversity of the limited dataset.

2. Pre-trained DenseNet

A pre-trained DenseNet can be used as the feature extractor.

The model may already recognize:

- Edges
- Leaf textures
- Shapes
- Spots
- Color patterns
- Disease-related visual structures

3. Dense Connections

The major characteristic of DenseNet is that each layer receives the feature maps from **all preceding layers within a dense block**.

Conceptually:

Layer 1 → Layer 2 → Layer 3 → Layer 4

But DenseNet creates connections such as:

Layer 1 → Layer 2, Layer 3, Layer 4

Layer 2 → Layer 3, Layer 4

Layer 3 → Layer 4

Thus, previously learned features are directly available to later layers.

4. Feature Reuse

Dense connections provide several advantages:

- Earlier features can be reused by later layers.
- Redundant feature learning is reduced.
- Important low-level features are preserved.
- Gradient flow is improved.
- The network can learn efficiently with fewer parameters.
- Useful for limited datasets.

For example, an early layer may detect **edges**, a middle layer may detect **leaf texture**, and a deeper layer may detect **disease spots**. Dense connections allow deeper layers to access these earlier features.

5. Classification Layer

Since there are five disease categories, the original classification layer is replaced with a new layer containing **5 output neurons**.

A **Softmax** activation is used.

Example classes:

1. Healthy
2. Leaf Blight
3. Leaf Rust
4. Powdery Mildew
5. Bacterial Spot

Model Training

- Freeze most pre-trained DenseNet layers initially.
- Train the new classification head.
- Gradually unfreeze selected deeper layers.

- Fine-tune using a low learning rate.
- Use **categorical cross-entropy loss**.
- Use **Adam** or another suitable optimizer.

Conclusion

DenseNet is suitable because dense connections encourage feature reuse and efficient gradient propagation. This makes the model effective for plant disease classification when only a limited number of training images are available.

3. A university wants to predict a student's future academic performance using their weekly attendance, assignment marks, internal assessment marks, and previous examination scores collected over several weeks. Design an LSTM-based model that can process this sequential information and predict the student's performance category.

Answer:

An **LSTM (Long Short-Term Memory)** network is suitable because student academic data is collected over several weeks and the information has a sequential relationship. The model can learn how previous academic behavior affects future performance.

Input Features

For every week, the model can receive:

- Weekly attendance percentage
- Assignment marks
- Internal assessment marks
- Previous examination score

For example:

Week 1 → Week 2 → Week 3 → Week 4 → ... → Week N

Each week is represented as a feature vector:

$X_t = [\text{Attendance}, \text{Assignment}, \text{Internal Assessment}, \text{Previous Exam Score}]$

Proposed Architecture

Weekly Student Data → Normalization → LSTM Layer → Dropout → Dense Layer → Softmax → Performance Category

1. Data Preprocessing

- Collect weekly student records.
- Handle missing values.
- Normalize numerical features.
- Arrange the data chronologically.
- Divide data into training, validation, and testing sets.

2. LSTM Layer

The LSTM receives data one time step at a time.

It maintains:

- **Cell state:** stores important long-term information.
- **Hidden state:** represents information useful for current prediction.

The LSTM gates control what information should be:

- Remembered
- Forgotten
- Updated
- Passed to the next time step

This allows the model to understand long-term academic trends.

3. Dense Layer

The final LSTM representation is passed to a fully connected Dense layer.

A dropout layer can be added to reduce overfitting.

4. Output Layer

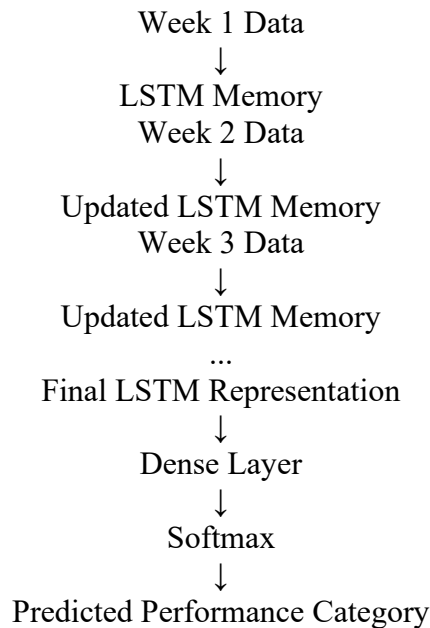
A Softmax output can classify students into categories such as:

- **Poor**
- **Average**

- **Good**
- **Excellent**

If there are four categories, the final layer contains **4 neurons**.

Example Workflow



Training

- Loss function: **Categorical Cross-Entropy**
- Optimizer: **Adam**
- Regularization: **Dropout**
- Evaluation metrics: Accuracy, Precision, Recall, and F1-score.

Conclusion

LSTM is suitable because academic performance depends not only on the current week's data but also on previous weeks' behavior. Its memory mechanism allows it to capture these temporal dependencies and predict the student's future performance category.

4. An organization wants to classify customer reviews as positive, negative, or neutral. Since the meaning of a word may depend on both the words appearing before and after it, design a Bidirectional RNN model for this application. Identify the major processing stages from input text to final sentiment prediction.

Answer:

A **Bidirectional Recurrent Neural Network (BiRNN)** is suitable for sentiment classification because it processes text in both directions. It considers the context from the words appearing before and after a particular word.

Proposed Architecture

Customer Review → Text Cleaning → Tokenization → Padding → Embedding → Forward RNN + Backward RNN → Concatenation → Dense Layer → Softmax → Sentiment

1. Text Preprocessing

The customer review is first processed by:

- Converting text to a consistent format.
- Removing unnecessary characters.
- Tokenizing the sentence into words or tokens.
- Converting words into numerical token IDs.
- Padding sequences to a fixed length.

Example:

"The product is very good"

becomes a sequence of numerical token representations.

2. Embedding Layer

The token IDs are converted into dense numerical vectors using an embedding layer. Words with similar meanings can have similar vector representations.

3. Bidirectional RNN

The BiRNN consists of two RNN processing directions:

Forward RNN:

Processes the sentence from left to right.

Backward RNN:

Processes the sentence from right to left.

For example:

The → product → is → very → good

The forward network learns information from the beginning to the end, while the backward network learns information from the end to the beginning.

4. Combining Information

The outputs from the forward and backward RNNs are combined, usually by concatenation. Therefore, each word can be represented using information from both:

- Previous words
- Following words

5. Dense Classification Layer

The combined representation is passed to a Dense layer.

Dropout may be included to reduce overfitting.

6. Output Layer

The final layer contains **3 neurons** because there are three sentiment classes:

- Positive
- Negative
- Neutral

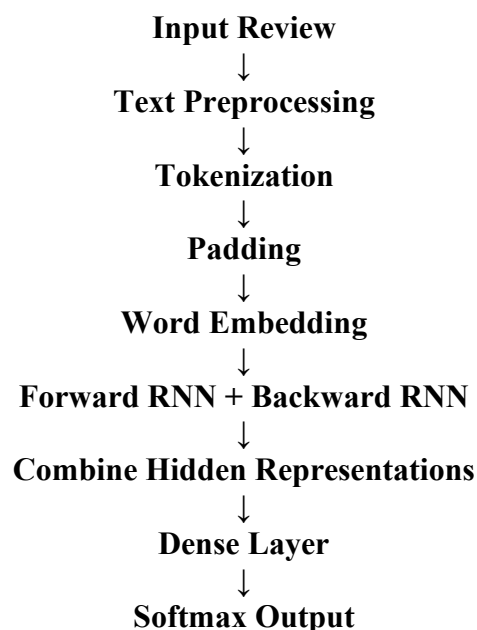
Softmax converts the outputs into probabilities.

Example:

Sentiment	Probability
Positive	0.82
Neutral	0.12
Negative	0.06

The model predicts **Positive**.

Major Processing Stages



↓
Positive / Negative / Neutral

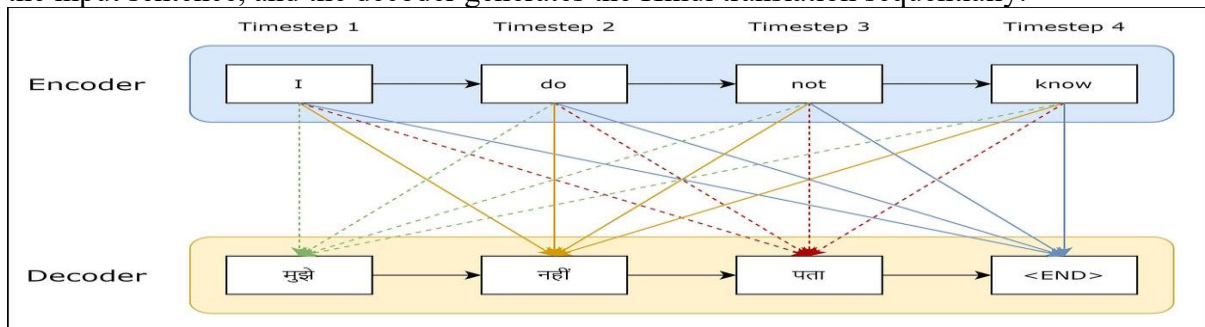
Conclusion

BiRNN improves sentiment classification by considering both past and future context. This is useful when the meaning of a word depends on the surrounding words in the review.

5. A software company wants to develop a system that translates English sentences into Hindi . The length of the input and output sentences may differ, and the system needs to understand the complete input sentence before generating the translated sentence. Design an Encoder–Decoder model and describe the role of the encoder, decoder, hidden representation, and output layer in the translation process.

Answer:

An **Encoder–Decoder architecture** is suitable for English-to-Hindi machine translation because the input and output sequences can have different lengths. The encoder understands the input sentence, and the decoder generates the Hindi translation sequentially.



Proposed Architecture

English Sentence → Tokenization → Embedding → Encoder → Hidden Representation → Decoder → Softmax → Hindi Sentence

An LSTM or GRU can be used for both the encoder and decoder.

1. Input Processing

The English sentence is converted into tokens.

Example:

"I am going to school"

The sentence is converted into numerical token IDs and then into word embeddings.

2. Encoder

The encoder reads the complete English sentence sequentially.

For example:

I → am → going → to → school

At each step, the encoder updates its hidden state.

The encoder learns:

- Word relationships
- Sentence structure
- Context
- Important semantic information

The final encoder representation summarizes the input sentence.

3. Hidden Representation

The hidden representation acts as an intermediate representation between the encoder and decoder.

It contains important information about the source sentence, including:

- Meaning
- Context
- Word relationships
- Sentence structure

This representation is passed to the decoder.

For longer sentences, an **attention mechanism** can be added so that the decoder can focus on relevant encoder states instead of relying only on one fixed representation.

4. Decoder

The decoder generates the Hindi translation one token at a time.

For example:

→ मैं → स्कूल → जा → रहा → हूँ →

At every step, the decoder uses:

- Its previous hidden state
- Previously generated word
- Encoder information/attention context

to predict the next Hindi word.

5. Output Layer

The decoder output is passed through a Dense layer followed by **Softmax**.

The Softmax layer calculates the probability of every possible Hindi word in the vocabulary.

For example:

Hindi Word Probability

मैं	0.70
वह	0.10
हम	0.05
आप	0.15

The word with the highest probability can be selected as the next output token.

6. Different Input and Output Lengths

One important advantage of the Encoder–Decoder architecture is that the input and output sequences do not need to have the same length.

For example:

English:

"I am going to the university."

Hindi:

"मैं विश्वविद्यालय जा रहा हूँ।"

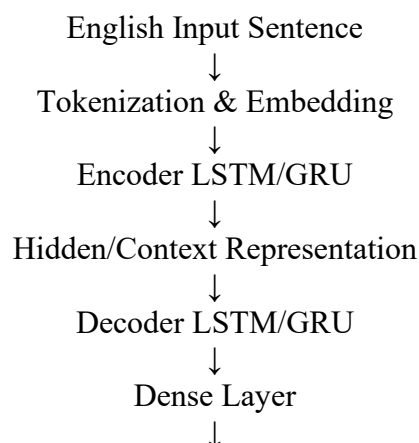
The number of words/tokens can differ between the two sequences.

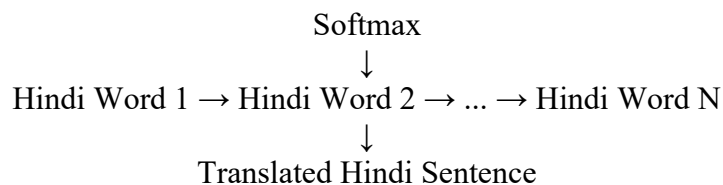
Training Process

During training:

1. English sentences are given to the encoder.
2. Corresponding Hindi sentences are given as target sequences.
3. The decoder learns to predict the next Hindi token.
4. The model minimizes translation error using categorical cross-entropy.
5. Teacher forcing can be used during training to provide the correct previous target token to the decoder.

Complete Architecture





Role of Major Components

Component	Role
Encoder	Reads and understands the complete English input
Hidden Representation	Stores the important contextual information of the input
Decoder	Generates the Hindi translation step by step
Output Layer	Calculates probabilities for possible next Hindi words
Softmax	Converts output scores into word probabilities
Attention	Helps the decoder focus on relevant parts of the input sentence

Conclusion

The Encoder–Decoder model is well suited for English-to-Hindi translation because it separates **understanding the source sentence** from **generating the target sentence**. The encoder captures the meaning of the English sentence, the hidden representation transfers this information to the decoder, and the decoder generates the Hindi translation sequentially. Attention can further improve performance for long sentences.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding & Requirement Analysis	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding; misinterprets problem context
Model Architecture & Design	30%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts
Application of Deep Learning Techniques	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution
Model Development & Implementation Strategy	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Performance Analysis & Justification	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis